

Springleik: A Case Study in Small Languages

Mark S. Williamsen

July 9, 2026

Abstract

Small languages arise for a variety of reasons in computing. A simple command interpreter can be useful in bringing up new hardware, particularly on small targets running embedded firmware. This leads naturally to a scriptable interface. Which can then evolve further into a primitive domain-specific language (DSL) that supports use cases in development, test, and even production. This work describes one path through the steps that lead from command interpreter to small language. Such languages can be very efficient with the resources they use including memory, processor bandwidth, and system services. Memory allocations can be static or dynamic, according to resources available and the details of the application. Additional features including self-documenting code and generation of baseline test outputs are easily implemented with minimal effort.